Algorithmes distribués
API du simulateur
processlibrary.jar

Classe Process

```
package ensisa.process;

import java.io.IOException;
import java.util.List;

/**
 * Models a process of a distributed system
 * This class is abstract and must be specialized to implement real processes
 */
public abstract class Process {

    /**
     * This process id
     * Starts at 0
     */
    public int id;

    /**
     * Number of processes
     */
    public int n;

    /**
     * List of process ids that can send messages to this process
     */
    public List<Integer> inNeighbors;

    /**
     * List of process ids to which this process can send messages
     */
    public List<Integer> outNeighbors;

    /**
     * Circuit mapped over current topology
     */
    public List<Integer> circuit;

    /**
     * Constructor
     *
     * @throws IOException
     */
    public Process() throws IOException;

    /**
     * Starts process
     */
    public void start() throws Exception;

    /**
     * Specific treatment realized by this process
     */
```

```
    protected abstract void run() throws Exception;

    /**
     * Sends a message to a neighbor process
     * to: id of process
     */
    public void send(int to, OutputMessage message) throws IOException;

    /**
     * Receives a message from a neighbor process.
     * receive is blocking.
     * result.senderProcessId: will contain the id of the sending process
     */
    public InputMessage receive() throws IOException;

    /**
     * Receives a message from the specified neighbor process.
     * receiveFrom is blocking.
     * from: id of the process from which a message is awaited
     */
    public InputMessage receiveFrom(int from) throws IOException;

    /**
     * Receives a message from the specified neighbor process and of the
     * specified type.
     * receiveFromType is blocking.
     * from: id of the process from which a message is awaited
     * type: type of the message awaited
     */
    public InputMessage receiveFromType(int from, int type) throws IOException;

    /**
     * Receives a message of the specified type.
     * receiveOfType is blocking.
     * type: type of the message awaited
     * result.senderProcessId: will contain the id of the sending process
     */
    public InputMessage receiveOfType(int type) throws IOException;

    /**
     * Receives a message of the specified type.
     * receiveOfType is blocking until time expiration.
     * type: type of the message awaited
     * timeout: time out in milliseconds
     * result is null if time expires
     */
    public InputMessage receiveOfType(int type, int timeout) throws
IOException;

        * Receives a message from a neighbor process or terminates this
process.
     *
     * @param terminate terminate must be a one element array, terminate[0] is
     * true if the process should end (from and message are undefined in this
     * case)
     * @return
     * @throws IOException
     */
    public InputMessage receiveOrTerminate(boolean[] terminate) throws
IOException;

    /**
     * Event oriented programming
     */
```

```
    /**
     * Registers a message handler of a receive event
     * handler is called when a message of the given type is received
     */
    public void onMessageReception(int type, MessageHandler handler);

    /**
     * Suspends execution until a message is received
     */
    public void waitReception() throws IOException;

}
```

Classe InputMessage

```
package ensisa.process;

import java.io.DataInputStream;

/**
 * Models a message received by a process
 */
public class InputMessage {
    // id of the process that sent this message
    public final int senderProcessId;
    // type of the message
    public final int type;

    InputMessage(int id, int type, byte[] bytes);

    /**
     * Extract an int value from the message content
     * The message content is handled as a stream
     */
    public int readInt();

    /**
     * Extract a string from the message content
     * The message content is handled as a stream
     */
    public String readString();

    /**
     * Access message content as a stream
     */

    public DataInputStream stream();


}
```

Classe OutputMessage

```
package ensisa.process;

import java.io.DataOutputStream;

/**
```

```java
 * Models a message sent by a process
 */
public class OutputMessage {
    // type of the message
    public final int type;

    /**
     * Creates an empty message with the given type
     * The message content is handled as a stream
     */
    public OutputMessage(int type);

    /**
     * Creates a message with the given type
     * value is inserted in the message content
     * The message content is handled as a stream
     */
    public OutputMessage(int type, int value);

    /**
     * Inserts value in the message content
     * The message content is handled as a stream
     */
    public void writeInt(int value);

    /**
     * Inserts the string in the message content
     * The message content is handled as a stream
     */
    public void writeString(String s);

    /**
     * Access the message content as a stream
     */
    public DataOutputStream stream();

}
```

Interface MessageHandler

```java
package ensisa.process;

import java.io.IOException;

@FunctionalInterface
public interface MessageHandler {
    void apply(InputMessage message) throws IOException;
}
```